

Count Islands

Given a binary matrix representing 1s as land and 0s as water, **return the number of islands**.

An island is formed by connecting adjacent lands 4-directionally (up, down, left, and right).

Example:

1	1	0	0
1	1	0	0
0	0	1	1
0	0	0	1

Output: 2

Intuition

Before determining how to find all the islands in a matrix, let's consider an input that contains only one island.

Matrix with just one island

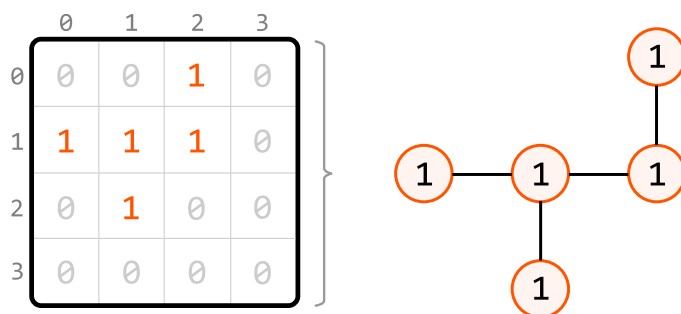
Consider the following matrix containing one island:

	0	1	2	3
0	0	0	1	0
1	1	1	1	0
2	0	1	0	0
3	0	0	0	0

When we iterate through the matrix starting from the top left, the first land cell we encounter is cell (0,

2). From here, we'd like to find the rest of the island.

The key observation is that we should be able to access every land cell on the same island by moving horizontally or vertically through neighboring land cells. This means that all 1s forming an island are connected either directly or indirectly through adjacent 1s. Conceptually, this is similar to a graph, where each cell is a node, and each connection to an adjacent cell is an edge:



This demonstrates that we can identify the rest of the island by performing a graph traversal algorithm. Most traversal algorithms will suit this purpose. In this explanation, we'll use DFS.

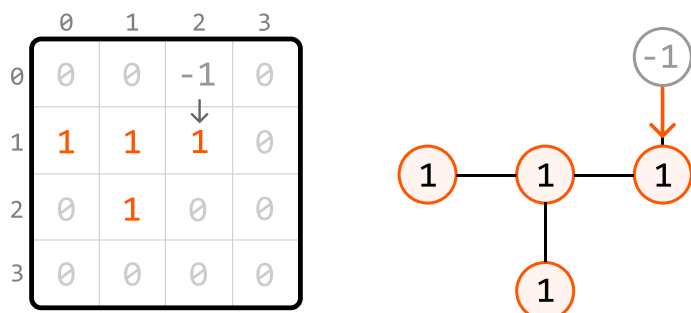
Depth-first search

For each cell we visit during the traversal, we need to mark that cell as visited to ensure it's not visited again. There are two ways to do this:

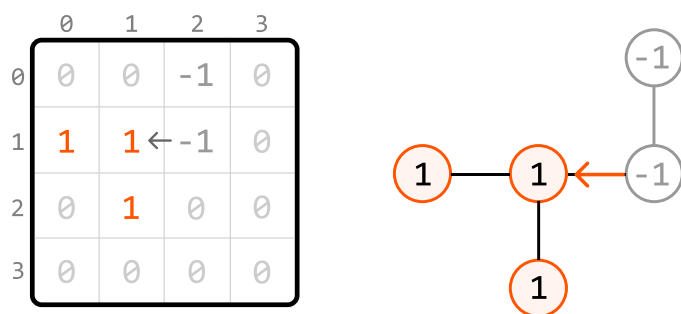
1. Use a separate data structure, such as a hash set, to keep track of the coordinates of visited cells.
2. Modify the matrix by **changing the value of a visited cell from 1 to -1**, ensuring it doesn't get revisited.

We'll proceed with the second option because it doesn't require the use of extra space.

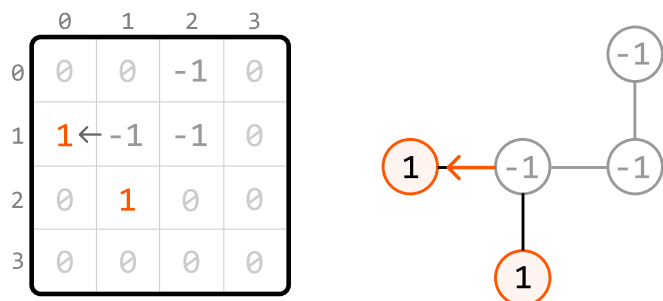
Now, let's begin DFS traversal. Mark the first cell, (0, 2), as visited by modifying its value to -1. Then, continue exploring by calling DFS on any neighboring land cells. Here, the only neighboring land cell is cell (1, 2):



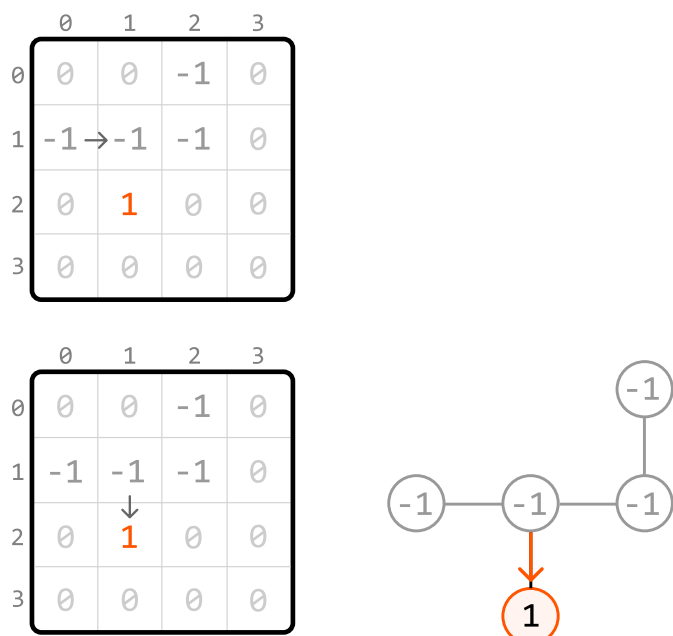
From cell (1, 2), we similarly mark it as visited and explore its neighboring land cells. Again, there's only one neighboring land cell: (1, 1). So, let's make a recursive DFS call to it:



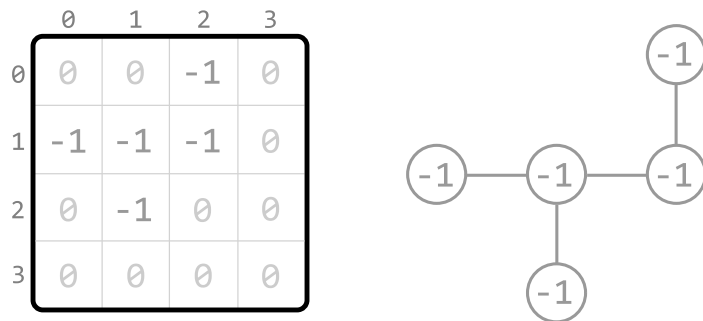
From cell (1, 1), mark it as visited and explore both of its neighboring land cells. Let's continue exploring from cell (1, 0) first:



At (1, 0), there's no neighboring land. So, the recursive process naturally goes back to cell (1, 1) to continue exploring any other neighboring land cells:



We've now finished exploring this island:

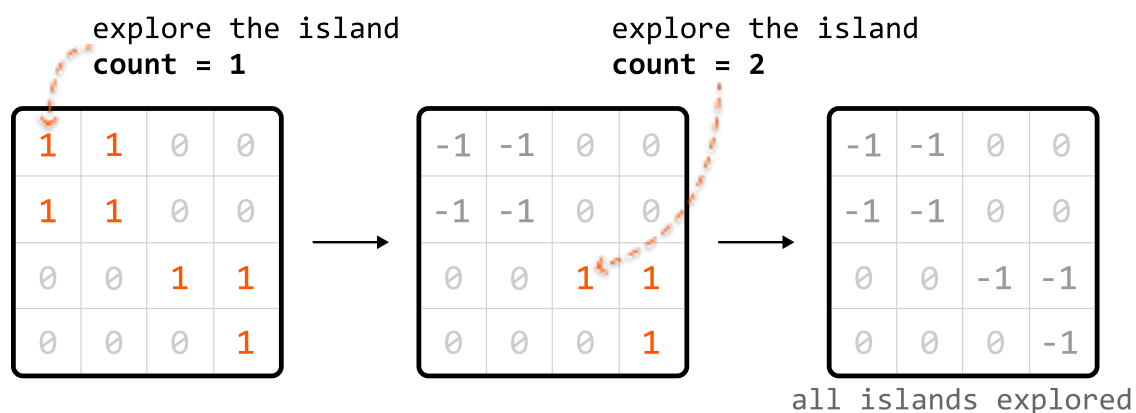


With this island completely explored, let's increment a variable count to indicate that one new island has been found. Now, let's consider the main problem where there could be multiple islands in the matrix.

Matrix with multiple islands

We can identify all islands using the following steps:

1. Search through the matrix, starting from cell (0, 0), until we find a land cell.
2. Upon encountering a land cell, explore its island using DFS, marking each land cell we encounter as visited (-1) to avoid visiting them again.
3. Increment count by 1, indicating the discovery of the island we just explored.
4. Keep searching the matrix for any unvisited land cells. When we find one, repeat steps 2 to 4.

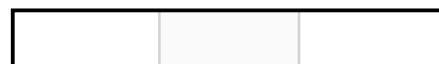


Implementation

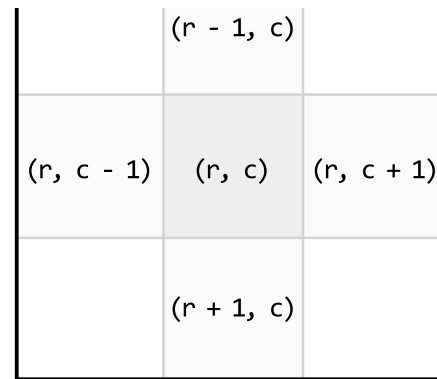
To traverse the matrix 4-directionally, we can use an array of **direction vectors**:

```
dirs = [(-1, 0), (1, 0), (0, -1), (0, 1)]
```

Each pair in the array represents the changes needed to move one step in a specific direction:



- **Up:** $(r, c) + \text{dirs}[0] = (r - 1, c)$
- **Down:** $(r, c) + \text{dirs}[1] = (r + 1, c)$
- **Left:** $(r, c) + \text{dirs}[2] = (r, c - 1)$
- **Right:** $(r, c) + \text{dirs}[3] = (r, c + 1)$



Python

JavaScript

Java

```
from typing import List
```



```
def count_islands(matrix: List[List[int]]) → int:
```

```
    if not matrix:
```

```
        return 0
```

```
    count = 0
```

```
    for r in range(len(matrix)):
```

```
        for c in range(len(matrix[0])):
```

```
            # If a land cell is found, perform DFS to explore the full island, and
            # include this island in our count.
```

```
            if matrix[r][c] == 1:
```

```
                dfs(r, c, matrix)
```

```
                count += 1
```

```
    return count
```

```
def dfs(r: int, c: int, matrix: List[List[int]]) → None:
```

```
    # Mark the current land cell as visited.
```

```
    matrix[r][c] = -1
```

```
    # Define direction vectors for up, down, left, and right.
```

```
    dirs = [(-1, 0), (1, 0), (0, -1), (0, 1)]
```

```
    # Recursively call DFS on each neighboring land cell to continue exploring the
    # island.
```

```
    for d in dirs:
```

```
        next_r, next_c = r + d[0], c + d[1]
```

```
        if is_within_bounds(next_r, next_c, matrix) and matrix[next_r][next_c] == 1:
            dfs(next_r, next_c, matrix)
```

```
def is_within_bounds(r: int, c: int, matrix: List[List[int]]) → bool:
```

```
    return 0 ≤ r < len(matrix) and 0 ≤ c < len(matrix[0])
```

Complexity Analysis

Time complexity: The time complexity of `count_islands` is $O(m \cdot n)$, where m denotes the number of rows and n denotes the number of columns. This is because each cell of the matrix is visited at most twice: once when searching for land cells in the `count_islands` function, and up to one more time during DFS.

Space complexity: The space complexity is $O(m \cdot n)$ mostly due to the recursive call stack during DFS, which can grow up to $m \cdot n$ in size.

Interview Tip

Tip: Check with an interviewer if modifications to the input are acceptable.

During DFS, we marked cells as visited by modifying the input directly. However, in some situations, input modification may not be desirable. As such, it's worth confirming this with the interviewer before making in-place modifications to the input.